Unità 2 – L'architettura della CPU

CPU, bus, registri e ciclo macchina

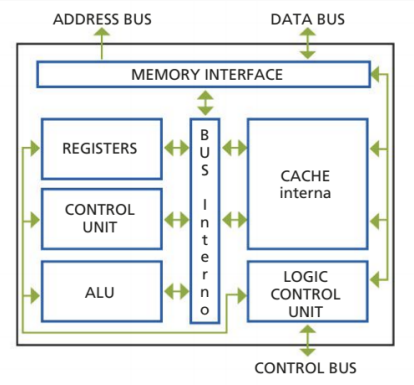
Francesco Gobbi

27 novembre 2025

I.I.S.S. Galileo Galilei – Ostiglia (MN)

Che cos'è la CPU

- **CPU** (*Central Processing Unit*): esegue le istruzioni dei programmi.
- Realizzata in un **microprocessore**, spesso **multicore**.
- L'**architettura** mantiene nel tempo gli stessi ruoli: controllo, calcolo, memoria interna e interfacce.



I blocchi fondamentali della CPU - Parte 1

La CPU è composta da più **unità funzionali**, ognuna con un ruolo specifico nell'elaborazione dei dati e nel controllo delle operazioni interne.

- **CU (Control Unit)**: coordina e gestisce le operazioni interne dei vari blocchi in base ai segnali provenienti dall'esterno e alle istruzioni da eseguire. È il "direttore d'orchestra" della CPU.
- **ALU (Arithmetic Logic Unit)**: esegue le operazioni aritmetiche e logiche (addizioni, sottrazioni, confronti, AND, OR, ecc.) richieste dall'unità di controllo.
- **REGISTERS** Piccole e velocissime aree di memoria interne che memorizzano dati, indirizzi o informazioni di stato durante l'esecuzione delle istruzioni. Il numero e la dimensione dei registri influiscono sulla velocità della CPU.
- **CACHE interna** Memoria ad accesso rapidissimo dove vengono conservate le istruzioni e i dati utilizzati più di frequente. È organizzata su più livelli (**L1**, **L2**, **L3**) per ottimizzare le prestazioni.

I blocchi fondamentali della CPU - Parte 2

- **LCU (Logic Control Unit)**: insieme di circuiti logici che convertono i segnali elettrici provenienti dall'esterno in segnali utilizzabili dall'unità di controllo, e viceversa. Permette la comunicazione tra CPU e periferiche di controllo.
- **MI (Memory Interface)**: gestisce le comunicazioni tra la CPU e la memoria, inviando e ricevendo indirizzi e dati tramite i bus esterni.
- **BUS interno** (o **CPU bus**): insieme di collegamenti elettrici che collegano tra loro i vari blocchi interni del microprocessore, permettendo lo scambio di dati, indirizzi e segnali di controllo.

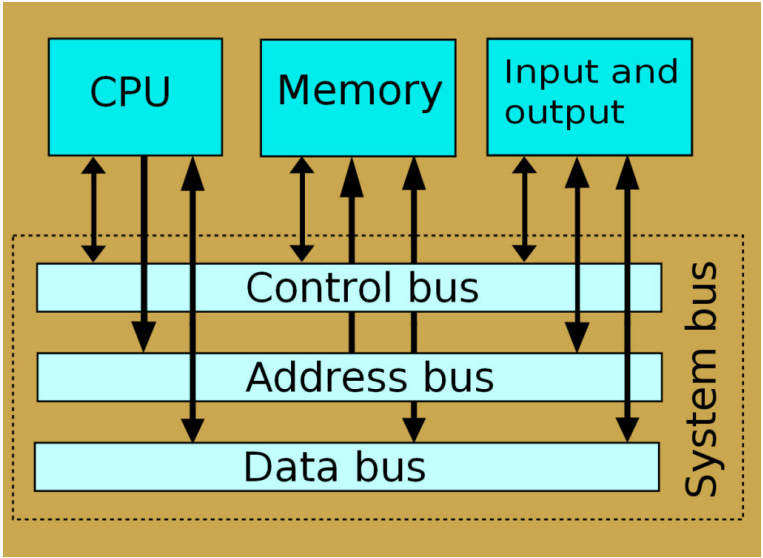
I bus del sistema

La **CPU** comunica con memoria e periferiche tramite tre bus che formano il **System Bus**:

- **Control bus**: trasporta i comandi e i segnali di controllo (*read*, *write*, clock, reset). Coordina i trasferimenti in base al **clock di sistema**.
- **Address bus**: trasporta gli indirizzi di celle o dispositivi a cui la CPU vuole accedere (lettura o scrittura). È **unidirezionale** dalla CPU verso la memoria/dispositivi.
- **Data bus**: trasporta i dati tra CPU e memoria/dispositivi; è **bidirezionale** (scrittura CPU→memoria, lettura memoria→CPU).

In molte architetture moderne **Address** e **Data bus** sono combinati in un unico bus *multiplexato* per ridurre la complessità circuitale.

I bus del sistema



Architettura a 32 e 64 bit

- **Larghezza di registri e bus**: più bit significano più dati elaborati per ciclo e più indirizzi di memoria gestibili.
- **CPU a 64 bit**: elabora parole da 64 bit e può indirizzare fino a 2^{64} celle di memoria virtuale.
- **Coerenza HW/SW**: per prestazioni ottimali devono essere allineati **CPU** $\leftrightarrow$ **Sistema Operativo** $\leftrightarrow$ **Software**.
- **Combinazioni tipiche**: CPU 32 bit + SO 32 bit; CPU 64 bit + SO 32 bit; CPU 64 bit + SO 64 bit (**migliore**).
- **Compatibilità**: programmi a 32 bit funzionano in genere su SO a 64 bit; non vale il contrario. Possibili incompatibilità con software che usa **driver** incorporati.

I registri della CPU

Un **registro** è una piccola e velocissima memoria interna alla CPU, utilizzata per **conservare temporaneamente dati, indirizzi e informazioni di controllo**.

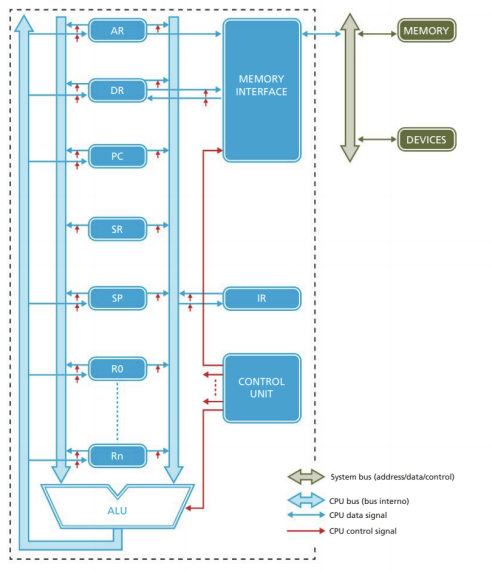
- I registri operano ad altissima velocità, direttamente collegati all'**unità di controllo** e all'**ALU**.
- La loro dimensione (8, 16, 32 o 64 bit) determina quante informazioni possono elaborare in un ciclo di clock.
- L'evoluzione delle CPU ha portato a un aumento sia del numero che della dimensione dei registri, con lo standard attuale a **64 bit**.
- Ogni registro ha una funzione specifica, ma tutti cooperano per garantire l'esecuzione corretta delle istruzioni.

I registri fondamentali sono presenti in tutte le CPU, anche se i nomi possono variare a seconda del produttore (Intel, AMD, ARM, ecc.). Altri registri specializzati possono contenere indici, contatori, puntatori o indirizzi base per operazioni complesse.

Tipi di registri e funzioni principali

- **AR (Address Register)**: memorizza gli indirizzi di memoria usati negli accessi, solo in direzione CPU $\rightarrow$ memoria.
- **DR (Data Register)**: conserva i dati trasferiti tra CPU e memoria, sia in lettura che in scrittura.
- **IR (Instruction Register)**: contiene l'**opcode** dell'istruzione da eseguire; l'unità di controllo lo decodifica per determinare l'operazione richiesta.
- **PC (Program Counter)**: registra l'indirizzo della prossima istruzione da eseguire, permettendo al programma di procedere in sequenza.
- **SR (Status Register)**: mantiene lo stato del processore tramite una serie di **flag** (zero, negativo, carry, overflow, ecc.) aggiornati dopo ogni operazione.
- **SP (Stack Pointer)**: punta alla cima dello **stack**, un'area di memoria LIFO dove vengono salvati indirizzi di ritorno e dati temporanei.
- **R0, ..., Rn (registri di lavoro)**: detti anche **registri generali**, conservano risultati parziali in ingresso e in uscita dall'ALU.

Schema dei registri nella CPU



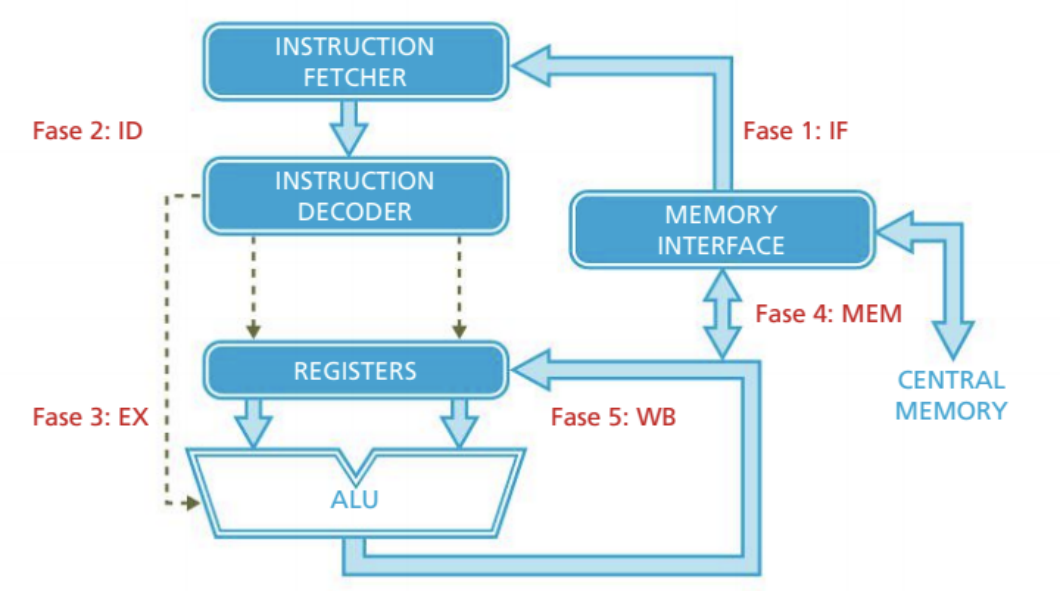
Il ciclo macchina (fetch–execute cycle)

La CPU esegue i programmi ripetendo una sequenza di fasi detta **ciclo macchina**:

1. **IF (Instruction Fetch)** — lettura dell'istruzione dalla memoria;
2. **ID (Instruction Decode)** — decodifica dell'istruzione e preparazione degli operandi;
3. **EX (Execution)** — esecuzione nell'ALU o in unità dedicate;
4. **MEM (Memory)** — eventuale accesso in lettura/scrittura alla memoria;
5. **WB (Write Back)** — scrittura del risultato e aggiornamento dei flag.

Queste fasi si ripetono in sequenza fino al termine del programma.

Il ciclo macchina (fetch–execute cycle)



Clock, cycle time e timeline

- Il **clock** scandisce il ritmo delle micro-operazioni: frequenza $= 1/t_{\text{CPU}}$.
- Il **cycle time** t_{CPU} è la durata minima di una micro-operazione sincronizzata.
- Esempio: $2\,\text{GHz} \Rightarrow t_{\text{CPU}} = 0{,}5\,\text{ns}$.
- In esecuzione sequenziale ogni istruzione occupa più fasi (IF, ID, EX, MEM, WB); la timeline mostra la collocazione delle fasi nel tempo.

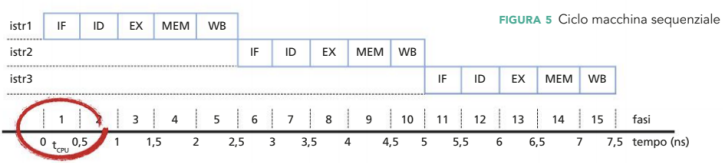


FIGURA 5 Ciclo macchina sequenziale

Dettaglio fasi: IF e ID (registri coinvolti)

Fase 1 — IF (Instruction Fetch)

- L'indirizzo della prossima istruzione è nel **PC** (Program Counter).
- Il PC viene copiato nell'**AR** (Address Register) per l'accesso in memoria.
- L'istruzione viene letta dalla memoria e caricata nell'**IR** (Instruction Register).
- Il **PC** viene incrementato per puntare all'istruzione successiva.
- Se l'istruzione richiede dati immediati, questi possono essere caricati nel **DR** (Data Register).

Fase 2 — ID (Instruction Decode)

- L'**IR** fornisce l'*opcode* all'unità di controllo, che **decodifica** l'operazione.
- Si preparano gli **operandi** caricando i registri di lavoro R0 . . . Rn e si attivano le microcircuiterie necessarie.

Dettaglio fasi: EX, MEM e WB

Fase 3 — EX (Execution)

- L'**ALU** (o un'unità funzionale dedicata) esegue l'operazione usando i registri di lavoro.
- Il risultato parziale può essere posto nel **DR** o in un registro generale.

Fase 4 — MEM (Memory)

- Se l'istruzione lo prevede, si effettua un **accesso in memoria**: lettura di un operando o scrittura del risultato.
- Non tutte le istruzioni richiedono questa fase.

Fase 5 — WB (Write Back)

- Il risultato finale viene scritto nel **registro** destinazione o in memoria.
- Il **SR** (Status Register) aggiorna i **flag** (Z, N, C, V, ...) in base all'esito dell'operazione.

Micro-operazioni e suddivisione del ciclo macchina

Gli attuali microprocessori non eseguono più ogni fase del ciclo macchina come un blocco unico: suddividono ciascuna fase in una serie di **micro-operazioni**, tutte di complessità paragonabile e sincronizzate dal clock.

- Ogni micro-operazione viene completata in **un ciclo di clock**.
- Una fase del ciclo macchina può essere composta da più micro-operazioni consecutive.
- L'ottimizzazione delle micro-operazioni consente una maggiore frequenza di clock e migliori prestazioni.

Un **ciclo macchina** è quindi una sequenza di fasi, ognuna composta da micro-operazioni. La durata di una micro-operazione è detta **cycle time** (t_{CPU}).

Clock, cycle time e tempo di esecuzione

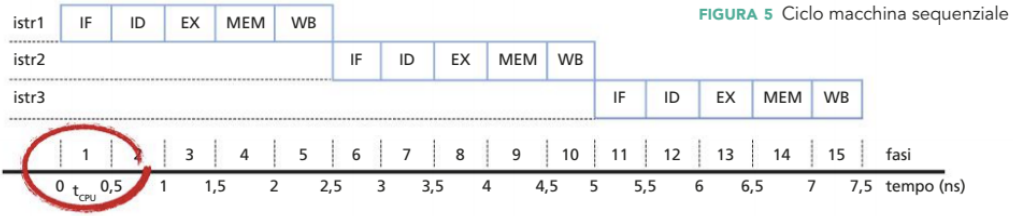


FIGURA 5 Ciclo macchina sequenziale

Clock, cycle time e tempo di esecuzione

La **frequenza di clock** della CPU è data da:

$$f = \frac{1}{t_{\text{CPU}}}$$

Dove t_{CPU} è la durata di una micro-operazione.

Esempio:

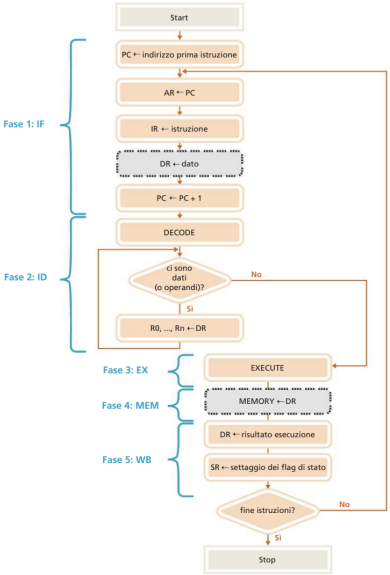
$$f = 2\,\text{GHz} \quad \Rightarrow \quad t_{\text{CPU}} = 0{,}5\,\text{ns}$$

Esecuzione sequenziale di tre istruzioni (senza pipelining):

- Ogni istruzione richiede 5 fasi del ciclo macchina.
- Tempo totale $= 3 \times 5 \times 0{,}5 = 7{,}5\,\text{ns}$.
- Ogni fase occupa un intervallo pari a t_{CPU}.

Ciclo macchina: schema a blocchi

FIGURA 6 Flowchart del ciclo macchina



La tecnica del pipelining

Il **pipelining** è una tecnica che permette alla CPU di elaborare più istruzioni in parallelo, sfruttando l'esecuzione contemporanea di micro-operazioni appartenenti a fasi diverse del ciclo macchina.

- Ogni fase del ciclo macchina (IF, ID, EX, MEM, WB) è gestita da una diversa **unità funzionale** specializzata.
- Le unità funzionano in modo **indipendente e parallelo**, come i reparti di una catena di montaggio.
- Quando la prima istruzione passa alla seconda fase, la fase iniziale è libera per l'istruzione successiva.
- Questo consente di ottenere un notevole incremento di prestazioni: la CPU esegue più istruzioni "sovrapposte" nel tempo.

La tecnica del pipelining

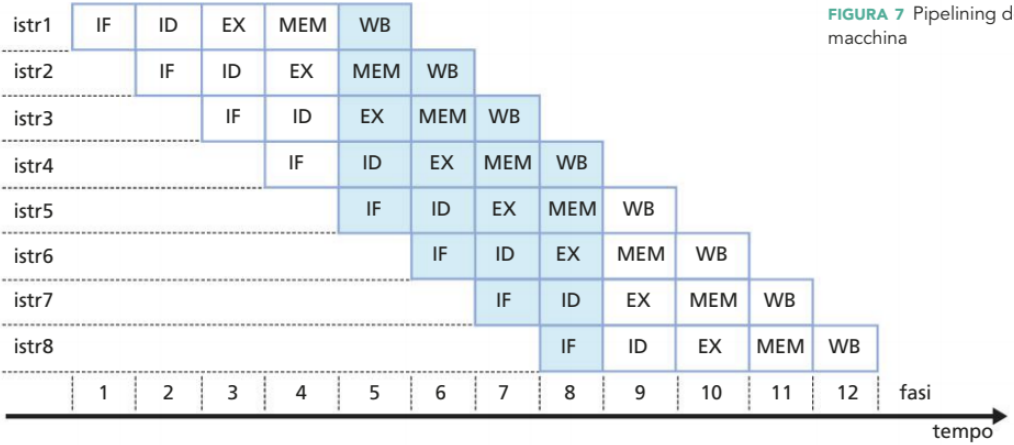


FIGURA 7 Pipelining del ciclo macchina

Esecuzione parallela delle istruzioni

Ogni fase del ciclo è gestita da una "conduttura" (**pipeline**) che elabora una fase di un'istruzione per volta. Dopo l'avvio, tutte le pipeline lavorano simultaneamente su istruzioni diverse.

- Esempio: 5 pipeline indipendenti elaborano un programma di 8 istruzioni.
- Dal tempo 5 al tempo 8, tutte le unità sono impegnate in fasi diverse: la pipeline lavora a pieno regime.
- In 12 fasi complessive vengono completate 8 istruzioni; senza pipelining, servirebbero 40 fasi (5×8).

Problemi di gestione della pipeline

Il pipelining è molto efficace, ma può incontrare difficoltà quando le istruzioni non sono del tutto indipendenti.

Principali problemi:

- **Dipendenze tra istruzioni**: un'istruzione deve aspettare il risultato di quella precedente prima di poter proseguire.
- **Salti di programma**: in caso di salto (if, cicli, condizioni), la pipeline deve essere svuotata e riempita di nuovo, rallentando l'esecuzione.

Soluzioni ai problemi della pipeline

Per ridurre gli stalli e migliorare le prestazioni, le CPU moderne utilizzano:

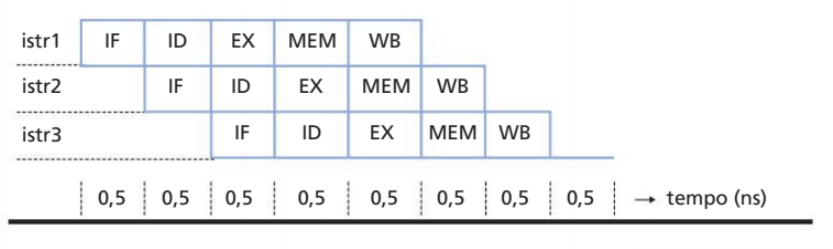
- **Registri più veloci** per rendere subito disponibili i risultati.
- **Pipeline multiple** per eseguire più istruzioni in parallelo.
- **Branch prediction** per "indovinare" il percorso del programma e ridurre i salti.
- **Micro-fasi più brevi** per aumentare la velocità di ogni operazione.

Nelle CPU multicore ogni core ha una pipeline propria, riducendo ulteriormente questi problemi.

Esempi pratici: confronto tra pipeline a 3 e 5 elementi

Caso A: pipeline a 3 elementi

- Ogni fase dura 0,5 ns.
- 3 istruzioni completate in 7 fasi $\Rightarrow T = 3{,}5$ ns.
- Frequenza di esecuzione:
  $\frac{3}{3{,}5 \times 10^{-9}} = 857{,}1$ MIPS = Million Instructions Per Second.

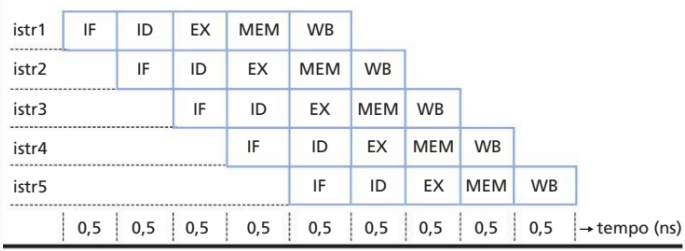


***MIPS = Million Instructions Per Second**, è l'unità di misura della frequenza di esecuzioni delle istruzioni effettuate da un microprocessore.*

Esempi pratici: confronto tra pipeline a 3 e 5 elementi

Caso B: pipeline a 5 elementi

- 5 istruzioni completate in 9 fasi $\Rightarrow T = 4{,}5$ ns.
- Frequenza di esecuzione: $\frac{5}{4{,}5 \times 10^{-9}} = 1{,}111$ MIPS.
- Miglioramento effettivo: $1{,}111 - 857{,}1 = 253{,}9$ MIPS.



Per ottenere la stessa velocità con 3 elementi, il tempo di ogni fase deve ridursi a:

$$t_f = 0{,}39 \text{ ns}$$

Esercizio guidato: analisi di una pipeline

Una CPU utilizza una pipeline di 4 fasi (IF, ID, EX, WB). Ogni fase dura 0,4 ns.

Determina:

1. il tempo necessario per completare le prime 4 istruzioni con il pipelining;
2. la frequenza effettiva in MIPS ottenuta dalle 4 istruzioni;
3. il tempo necessario per completare 10 istruzioni
   - con il pipelining;
   - senza pipelining.

Svolgimento

- **Tempo di riempimento della pipeline** Ci sono 4 fasi, quindi per riempire la pipeline servono:

$$T_{\text{riemp}} = 4 \times 0{,}4 = 1{,}6\,\text{ns}$$

Esercizio guidato: analisi di una pipeline

- **Prime 4 istruzioni con pipelining** Il tempo totale per n istruzioni in una pipeline di m fasi è:

$$T(n) = (m + n - 1)\, t_{\text{CPU}}$$

Qui $m = 4,\ n = 4,\ t_{\text{CPU}} = 0{,}4\,\text{ns}$:

$$T(4) = (4 + 4 - 1) \times 0{,}4 = 7 \times 0{,}4 = 2{,}8\,\text{ns}$$

- **Frequenza effettiva con le 4 istruzioni**

$$f_{\text{eff}} = \frac{4 \text{ istruzioni}}{2{,}8 \times 10^{-9}\,\text{s}} \approx 1{,}43 \times 10^{9} \text{ IPS} \approx 1430 \text{ MIPS}$$

Esercizio guidato: analisi di una pipeline

- **10 istruzioni con pipelining**

$$T(10) = (4 + 10 - 1) \times 0{,}4 = 13 \times 0{,}4 = 5{,}2\,\text{ns}$$

  Frequenza effettiva:

$$f_{\text{eff}}(10) = \frac{10}{5{,}2 \times 10^{-9}} \approx 1{,}92 \times 10^9 \text{ IPS} \approx 1920 \text{ MIPS}$$

- **10 istruzioni senza pipelining** Ogni istruzione deve fare 4 fasi in sequenza:

$$T_{\text{nopipe}}(10) = 10 \times 4 \times 0{,}4 = 16\,\text{ns}$$

Conclusione: con 10 istruzioni il pipelining riduce il tempo da 16 ns a 5,2 ns, con un miglioramento di circa $\frac{16}{5{,}2} \approx 3$ volte rispetto all'esecuzione sequenziale.

Linguaggio macchina e architetture

Il **linguaggio macchina** è l'insieme delle istruzioni che la CPU può comprendere ed eseguire direttamente. Ogni **istruzione** è rappresentata come una sequenza di **bit**, divisi in **campi** che indicano:

- l'**operazione** da svolgere (**opcode**);
- gli **operandi** o i loro indirizzi;
- eventuali **modi di indirizzamento**.

Ogni costruttore di CPU mantiene la **retrocompatibilità**: le istruzioni dei processori più vecchi possono essere eseguite anche su quelli più recenti.

Linguaggio macchina e architetture

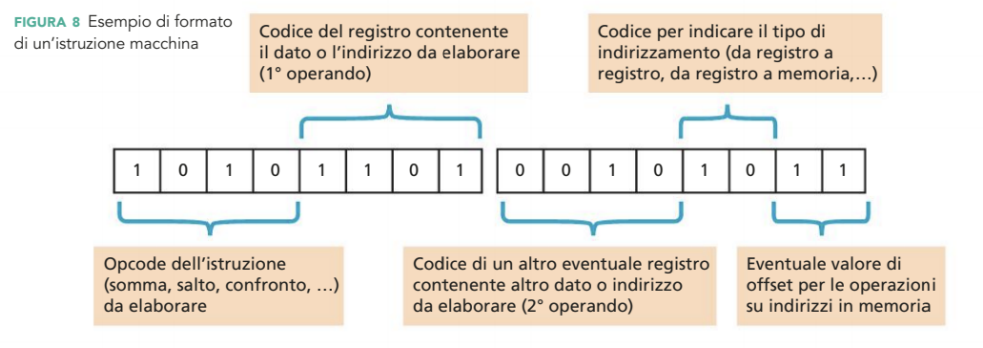


FIGURA 8 Esempio di formato di un'istruzione macchina

Architetture delle istruzioni

Le architetture si differenziano in base al **numero di operandi** che un'istruzione può contenere. Possiamo avere:

- **architettura a un operando**;
- **architettura a due operandi**;
- **architettura a tre operandi**.

Ogni scelta influenza:

- la **semplicità** del linguaggio macchina;
- la **quantità di bit** da usare per ogni istruzione;
- la **velocità** con cui vengono elaborate le istruzioni.

Architettura a un operando

Una istruzione contiene:

Opcode	Operando

Il campo **opcode** è presente e specifica il **comando** che la CPU deve eseguire.
L'**operando** rappresenta direttamente il dato o l'indirizzo da cui prendere il dato.
L'altro operando necessario viene preso da un **registro interno** già predisposto dalla CPU.

Vantaggi:

- molti bit disponibili per l'operando o il suo indirizzo.

Svantaggi:

- richiede operazioni intermedie: bisogna prima preparare in un registro il secondo operando.

Architettura a due operandi

Una istruzione contiene: Una istruzione contiene:

Opcode	Operando1/Risultato	Operando2

Il primo campo contiene inizialmente il **primo operando**, ma alla fine dell'operazione conterrà il **risultato**.

Vantaggi:

- permette di elaborare subito entrambi gli operandi;
- richiede meno istruzioni rispetto alla struttura a 1 operando.

Svantaggi:

- bisogna "salvare" il primo operando se serve riutilizzarlo.

Architettura a tre operandi

Una istruzione contiene: Una istruzione contiene:

Opcode	Risultato	Operando1	Operando2

Il campo **Risultato** è separato, quindi gli operandi originali non vengono sovrascritti.

Vantaggi:

- codifica chiara: operandi e risultato sono tutti **espliciti**;
- più facile da programmare.

Svantaggi:

- servono più bit per rappresentare l'istruzione;
- istruzioni tendenzialmente più lunghe.

Architetture CISC e RISC

Fin dai primi microprocessori si sono sviluppate due grandi filosofie progettuali:

- **CISC** — *Complex Instruction Set Computer*
- **RISC** — *Reduced Instruction Set Computer*

Queste due famiglie differiscono principalmente per:

- numero e complessità delle istruzioni;
- complessità della **Control Unit**;
- tempo di esecuzione delle istruzioni;
- facilità di implementazione del **pipelining**.

Architettura CISC

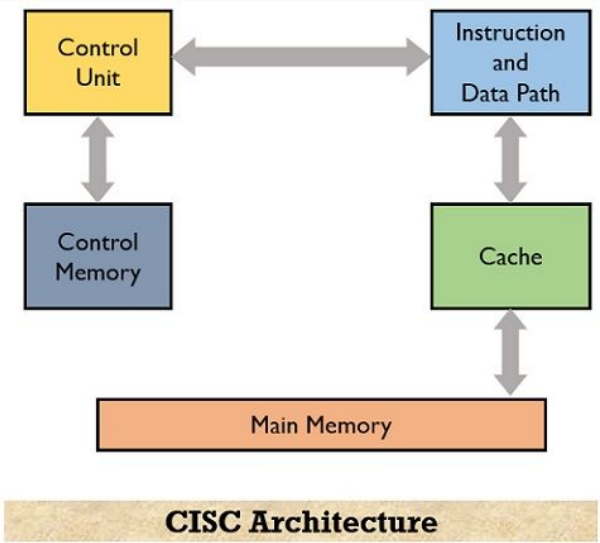
La filosofia delle CPU **CISC** è: **molte istruzioni, anche molto complesse**, per fare in un solo comando quello che altrimenti richiederebbe più istruzioni.

Caratteristiche principali:

- grande numero di istruzioni, anche molto ricche e articolate;
- istruzioni più lente (alto t_{CPU}), ma a volte più "potenti";
- Control Unit complessa: richiede microprogrammazione e firmware;
- frequenza di clock più limitata.

TDP: La decodifica complessa aumenta il **TDP** (*Thermal Design Power*). Più complessa è la Control Unit, maggiore è il calore generato — per questo storicamente le CPU CISC scaldavano di più.

Architettura CISC



Architettura RISC

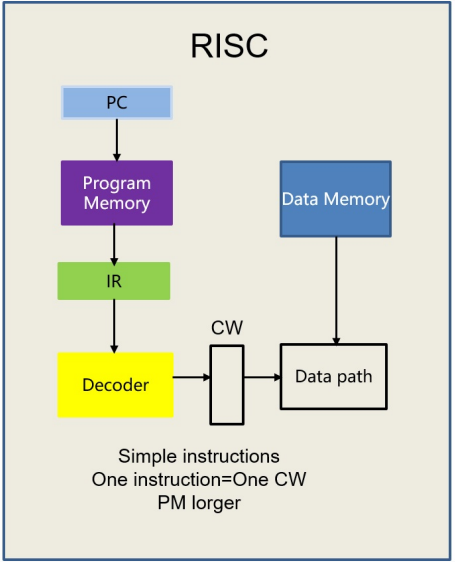
La filosofia delle CPU **RISC** è: **poche istruzioni, semplici e molto veloci**, da combinare tra loro.

Caratteristiche principali:

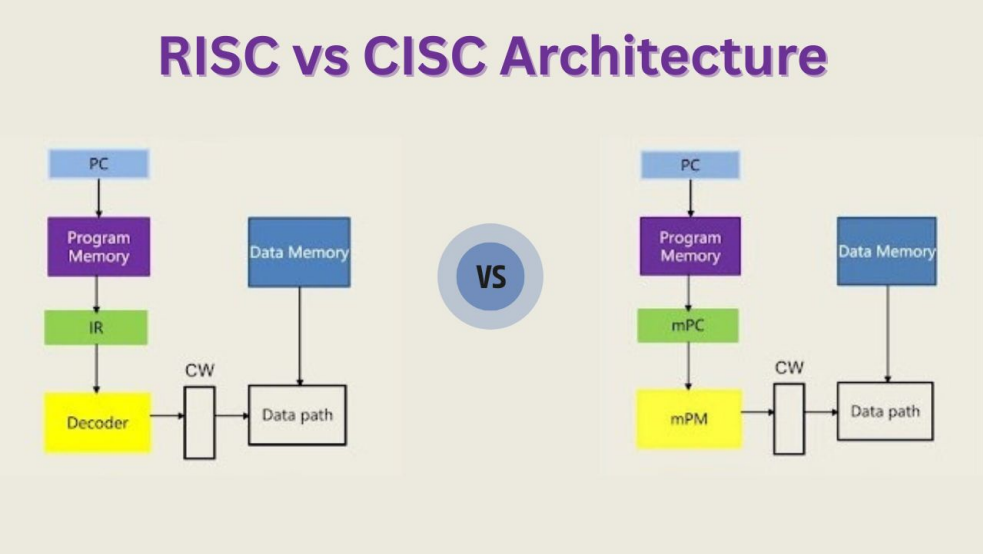
- set di istruzioni molto ridotto;
- istruzioni tutte della stessa lunghezza (standardizzate);
- esecuzione estremamente veloce (basso t_{CPU});
- Control Unit semplice e leggera;
- facilità di implementare un pipelining profondo.

Curiosità: - Le architetture RISC sono alla base di **ARM**, usate oggi in smartphone, tablet e nei Mac con Apple Silicon. - Sono molto efficienti dal punto di vista energetico: consumano meno e generano meno calore rispetto alle CISC tradizionali.

Architettura RISC



CISC vs RISC


RISC vs CISC Architecture
PC
Program Memory
IR
Decoder
CW
Data Memory
Data path
VS
PC
Program Memory
mPC
mPM
CW
Data Memory
Data path

CISC vs RISC: confronto moderno

Oggi la distinzione CISC/RISC è molto meno netta rispetto al passato.

CPU CISC moderne (es. Intel, AMD):

- all'esterno hanno istruzioni CISC tradizionali;
- **all'interno le convertono in micro-ops RISC**, cioè piccole operazioni semplici eseguite in pipeline;
- la pipeline è quindi più simile a una architettura RISC.

CPU RISC moderne (es. ARM):

- mantengono un set semplice e omogeneo;
- ottimizzano drasticamente consumi e prestazioni;
- eccellenti per dispositivi portatili e sistemi ad alta efficienza.

Curiosità: Molti processori Intel e AMD trasformano anche istruzioni complesse (CISC) in gruppi da 2–4 micro-istruzioni molto semplici, eseguite ad alta velocità.

RISC e pipelining

Le architetture RISC sono progettate per sfruttare al meglio il **pipelining**.

Perché RISC è ideale per il pipelining?

- istruzioni tutte della stessa lunghezza decodifica velocissima;
- istruzioni semplici suddivisione chiara in micro-operazioni;
- ridotta complessità della Control Unit meno stalli;
- pipeline più profonde e frequenze di clock più alte.

Curiosità:

- Nei processori moderni, i core "Efficiency" (Apple, Intel, ARM) sono spesso RISC-like.
- RISC e pipelining sono centrali nel design delle CPU multicore attuali.

Evoluzione dei microprocessori: tappe fondamentali

Fin dalla fine degli anni '60 i microprocessori hanno seguito una crescita rapida. Qui sono riportati i modelli più rappresentativi:

- **Intel 4004 (1971)** — 4 bit, 740 kHz Primo microprocessore commerciale al mondo.
- **MOS 6502 (1975)** — 8 bit, 2 MHz Usato nei primi home computer (Apple II, Commodore VIC-20).
- **Intel 8086 (1978)** — 16 bit, 10 MHz Base dell'architettura x86 ancora oggi utilizzata.
- **Intel Pentium (1993)** — 32 bit, 60–80 MHz Introdotte pipeline avanzate e cache L1/L2 integrate.
- **AMD Athlon64 (2003)** — 64 bit Prima architettura x86-64 realmente diffusa.
- **CPU moderne (2020–2025)** — 64 bit multicore Da 4 a 64 core, frequenze fino a 5 GHz, ampie cache, tecniche di risparmio energetico avanzate.

Caratteristiche principali di un microprocessore

Quando si confrontano due processori, è importante osservare:

- **Frequenza (GHz)** — velocità a cui opera la CPU.
- **Numero di core** — più core = più attività in parallelo.
- **Thread** — processi eseguibili contemporaneamente (HyperThreading, SMT).
- **Cache L1, L2, L3** — memorie interne molto veloci.
- **TDP (Thermal Design Power)** — quantità massima di calore da dissipare.
- **Litografia (nm)** — dimensione dei transistor: più piccoli più efficienza.
- **IPC (Instructions per Clock)** — quante istruzioni eseguite a parità di frequenza.

Cos'è il DIE di un microprocessore?

Il **DIE** è la parte fisica del processore:

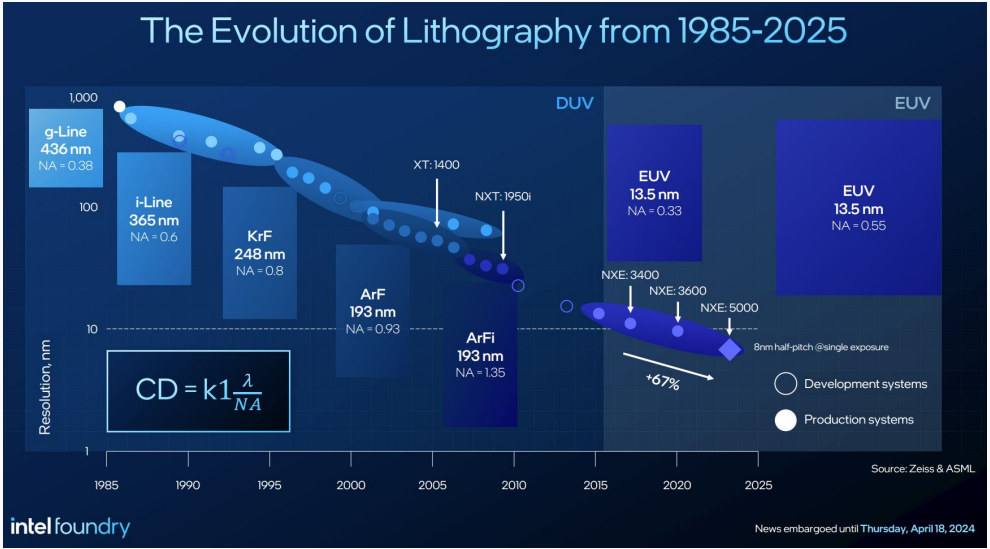
- una sottile piastra di silicio su cui sono "incisi" i transistor;
- contiene core, cache, controller di memoria e altre unità logiche;
- viene inserito in un **package** che permette il contatto con la scheda madre.

Più è grande il DIE, più componenti può contenere, ma anche più elevato sarà il consumo e il calore generato.

Oggi, grazie alle litografie avanzate, è possibile inserire:

- molti core,
- cache molto grandi,
- GPU integrate ad alte prestazioni.

Evoluzione della litografia dei microprocessori



Intel: tecnologie e direzione attuale (2025)

Intel rimane uno dei principali produttori di CPU per PC e notebook.

Tecnologie chiave nel 2025:

- **Architettura ibrida P-core / E-core** (Performance ed Efficiency cores) ottimizza prestazioni e consumi.
- **HyperThreading** per aumentare il numero di thread elaborabili.
- **Processo produttivo Intel 4 / Intel 3** con transistor più piccoli ed efficienti.
- **Grafica integrata Xe** più potente, utile per notebook.
- **AI accelerators integrati** per modelli di IA locale (NPU).

Le CPU moderne Intel sono progettate per:

- ottime prestazioni single-core,
- buone capacità di multitasking,
- riduzione dei consumi nei notebook.

Intel: tecnologie e direzione attuale (2025)

Intel® Core™ Ultra Processors (HX-SKUs)

Processor Number	Intel® Core™ Ultra 9 Processor 285HX	Intel® Core™ Ultra 9 Processor 275HX	Intel® Core™ Ultra 7 Processor 265HX	Intel® Core™ Ultra 7 Processor 255HX	Intel® Core™ Ultra 5 Processor 245HX	Intel® Core™ Ultra 5 Processor 235HX
Processor Cores (P-cores + E-cores)[1]	24 (8+16)	24 (8+16)	20 (8+12)	20 (8+12	14 (6+8)	14 (6+8)
Processor Threads	24	24	20	20	14	14
Intel® Smart Cache (LLC)	36 MB	36 MB	30 MB	30 MB	24 MB	24 MB
P-core Max Turbo Frequency[3]	Up to 5.5 GHz	Up to 5.4 GHz	Up to 5.3 GHz	Up to 5.2 GHz	Up to 5.1 GHz	Up to 5.1 GHz
E-core Frequency	Up to 2.8 GHz	Up to 2.7 GHz	Up to 2.6 GHz	Up to 2.4 GHz	Up to 3.1 GHz	Up to 2.9 GHz
Graphics Max Frequency	Up to 2.0 GHz	Up to 1.9 GHz	Up to 1.9 GHz	Up to 1.85 GHz	Up to 1.9 GHz	Up to 1.8 GHz
Processor Graphics	Intel® Graphics					
Total PCIe Lanes, Thunderbolt™ Technology, Wi-Fi	4 PCIe Gen 5 + 4 PCIe Gen 4 Lanes 2 Integrated Thunderbolt™ 4 Ports + Discrete Thunderbolt™ 5 Integrated Wi-Fi 6E + Discrete Wi-Fi 7[9]					
Max Memory Speed[10]	DDR5 - 6400 LP5x - 8400 LP5/5x CAMM2					
Maximum Memory Capacity/Rank	96 GB (DDR5) 64 GB (LP5/x)					
Processor Base Power	55 W					
Maximum Turbo Power	160 W					

AMD: tecnologie e direzione attuale (2025)

AMD è molto competitiva grazie all'architettura **Zen**.

Punti forti delle CPU AMD attuali:

- **Architettura Zen 4/Zen 5** con elevato IPC.
- **Chiplet design**: più moduli uniti sul package costi ridotti ed efficienza maggiore.
- **SMT** (Simultaneous Multithreading) come alternativa all'HyperThreading.
- Ottimo rapporto tra prestazioni e consumi.
- **Cache 3D V-Cache** nei modelli gaming.

Le CPU AMD nel 2025 eccellono in:

- multitasking pesante,
- lavoro professionale,
- gaming (grazie alla V-Cache),
- sistemi desktop ad alte prestazioni.

AMD: tecnologie e direzione attuale (2025)

Processor model						
AMD Ryzen™ AI Max+ AMD Ryzen™ AI Max+ PRO 395	16 cores 32 threads	up to 5.1 GHz max boost	80 MB Cache	50 Peak TOPS	45-120 W cTDP	40 Graphics Cores
AMD Ryzen™ AI Max AMD Ryzen™ AI Max PRO 390	12 cores 24 threads	up to 5.0 GHz max boost	76 MB Cache	50 Peak TOPS	45-120 W cTDP	32 Graphics Cores
AMD Ryzen™ AI Max AMD Ryzen™ AI Max PRO 385	8 cores 16 threads	up to 5.0 GHz max boost	40 MB Cache	50 Peak TOPS	45-120 W cTDP	32 Graphics Cores
AMD Ryzen™ AI Max PRO 380	6 cores 12 threads	up to 4.9 GHz max boost	22 MB Cache	50 Peak TOPS	45-120 W cTDP	16 Graphics Cores

Available **Q1-Q2 2025**

*See endnote GD-150, GD-243

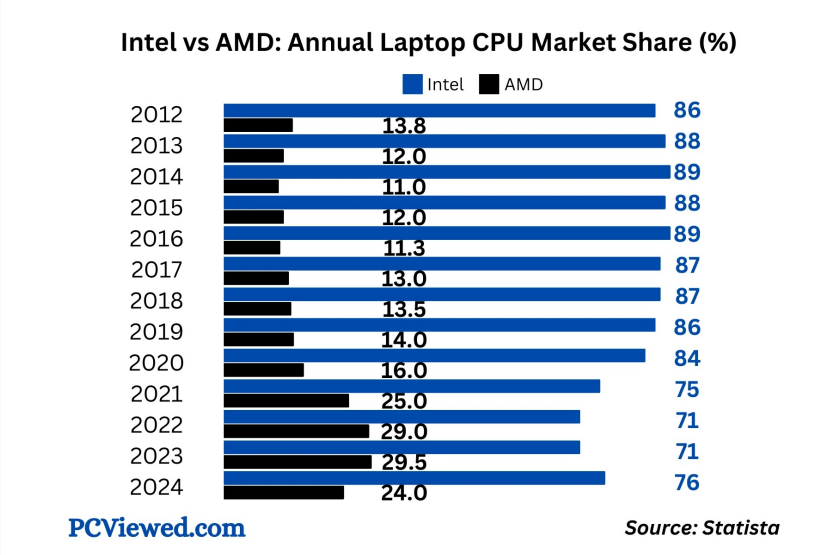
Intel vs AMD: confronto generale

Intel:

- Prestazioni single-core molto elevate.
- Tecnologia ibrida P-core/E-core.
- Consumi ottimizzati per notebook.

AMD:

- Architettura Zen molto efficiente.
- Elevate prestazioni multicore.
- Chiplet design e 3D V-Cache.

In sintesi: Intel è molto forte nei laptop e nelle prestazioni per core; AMD eccelle nella produttività, nel multicore e nel gaming di fascia alta.

Intel vs AMD: confronto generale